SEGUNDO LABORATORIO

Etapa 2: Refinando Registro (en el mismo módulo registro.py)
Objetivo: Construir dos clases hijas de Registro según tengamos un problema de clasificación o de regresión

1. **Clase RegistroClasificacion**
   - Los parámetros del constructor deben ser un List de números reales para la invocación del constructor padre y una cadena de caracteres como atributo **objetivo**.
   - Redefine el método normalizar (reusa el de la clase padre) para que ahora devuelva un RegistroClasificacion.
   - Redefinir el método __repr__ para visualización (reusa el de la clase padre).

2. **Clase RegistroRegresion:**
   - Las mismas funciones que RegistroClasificacion pero en el constructor el **objetivo** debe ser un valor real.
   - La redefinición de normalizar no sería necesaria para ambas clases si cambiamos el método normalizar de la clase padre haciendo que devuelva un objeto de la misma clase del objeto que invoca mediante:
     nuevo_registro = self.__class__(nuevos_datos, self.objetivo)
     Esto nos asegura que si normalizar es invocado por un RegistroClasificacion o un RegistroRegresion devuelve un objeto de la clase correcta.

Etapa 3: Clases contenedoras (en el mismo módulo dataset.py)
Objetivo: Definir clases que contengan un conjunto de objetos de tipo Registro.
Normalmente se opta por la estructura List para almacenar los objetos porque permite una manipulación más eficiente. En este caso vamos a optar por diseñar una clase contenedora DataSet que va a ser "**abstracta**". Eso quiere decir que nunca se va a instanciar (nunca se va a crear un objeto de la clase DataSet), pero se usa en POO para un mejor diseño de clases. Nuestros problemas van a ser siempre de clasificación o regresión, por tanto, vamos a tener DataSetClasificacion y DataSetRegresion, pero la clase DataSet nos servirá para poner el código común a ambas. Cuando tengas todo el código pregúntale a la IA porque esta opción de diseño es mejor que poner una clase DataSet no abstracta.

1. **Clase DataSet (abstracta):**
   - Tendrá como atributos dos listas denominadas **registros** y **nombres_atributos**.
   - Tendrá un constructor que inicialice las dos listas **registros** y **nombres_atributos** vacíos.
   - Tendrá un método denominado set_cabeceras que recibirá una lista (de str) y asignará todos sus elementos, menos el último, a **nombres_atributos**.

- Tendrá un método abstracto (sin código) denominado **agregar_registro** que recibirá un objeto registro y su código será ***pass***. Con esto estamos obligando a que las clases hijas obligatoriamente tenga que implementar un método **agregar_registro**.
- Tendrá una función **calcular_min_max** que devuelve dos listas con los valores máximos y mínimos para los valores de cada índice de la lista **registros** del objeto de tipo DataSet que invoca al método. Si la lista **registros** contiene los registros de valores [3.4,5.7], [2.7,7.3] y [4.5,6.4], entonces debe devolver como lista de mínimos [2.7 (mínimo de los valores 0), 5.7 (mínimo de los valores 1] y como lista de máximos [4,5, 7.3] (máximos de los valores 0 y 1 respectivamente).
- Tendrá una función **crear_subconjunto** que reciba una lista de Registro y devuelva un objeto de tipo DataSet que tendrá como **registros** la lista pasada como argumento o parámetro de la función y como **nombres_atributos** el mismo que el objeto que invoca a la función.

2. **Clases DataSetClasificacion y DataSetRegresion** (heredan de DataSet)
   - Ambas clases implementen la función **agregar_registro** que añade un objeto a registros. Los métodos deben analizar que el objeto que van a añadir es del tipo de DataSet en el que está implementada la función **agregar_registro**.

Etapa 4: Clases de Factoría (en el mismo módulo factoria.py)
Objetivo: Definir clases para crear objetos de tipo DataSet a partir de la lectura de ficheros.

3. **Clase FactoriaCSV:**
   - Implementa dos funciones estáticas crear_dataset_clasificacion y crear_dataset_regresion para crear respectivamente objetos de las clases DataSetClasificacion y DataSetRegresion. Estas funciones deberán recibir un índice con la situación de la columna que contiene la etiqueta (para clasificación) o el valor (para regresión). Por defecto será la última (índice -1). Tenga en cuenta que la primera fila del fichero CSV contiene la lista de nombres de los atributos (**nombres_atributos**).
   -
4. **Clase FactoriaXLS:**
   - Igual que la anterior, pero a partir de un fichero Excel

Etapa 5: Modelos de Machine Learning (en el módulo modelos.py)
Objetivo: Definir clases para modelar clasificadores y regresores.

5. **Clase Modelo (abstracta):**

- Esta clase tendrá los métodos y datos para implementar cualquier modelo de clasificación/regresión.
- Esta clase tendrá un atributo de tipo Dataset denominado **datos_entrenamiento** inicializado a nulo.
- Un método entrenar que recibirá un argumento de tipo DataSet que es asignado a **datos_entrenamiento**.
- Un método predecir que recibe un objeto de tipo Registro cuya clase o valor debe predecir. Como ese método dependerá del modelo concreto, aquí no tendrá código.

6. **Clase Clasificador_kNN** (hereda de Modelo)
   - El constructor recibe como argumentos el valor de k y la distancia
   - Se debe redefinir el método predecir que debe invocar al método k_vecinos de la clase Registro usando **datos_entrenamiento.** Este método devolverá la etiqueta más común entre los vecinos del Registro que se da como argumento.

Ejecuta el clasificador kNN con los ficheros iris, wine y diabetes proporcionados. Prueba distintos k y distancias para encontrar la mejor combinación que maximice la bondad del clasificador kNN para cada dataset. Prueba algunas combinaciones de pesos con la distancia ponderada para ver si encuentras una selección de pesos que mejores a euclídea o manhattan.

¿Qué pasa cuando k vale 1? Pregunta a tu IA favorita por el concepto de overfitting (o sobreajuste) y cómo se puede resolver. Dese cuenta de que en esta práctica no se pide implementar todavía ni normalización ni validación que se verán en el siguiente laboratorio.